CSL6010-Cybersecurity Major Project Report

Eyes on Your Typing Analysis and CASOM Defense

Team of 7 Members

Source Code Repository: `https://github.com/AzDevops143/CSIEEE`

June 14, 2026

Source Code Repository:
`https://github.com/AzDevops143/CSIEEE`

Contents

1 Introduction

The rapid growth of Augmented Reality (AR) and Virtual Reality (VR) technologies has introduced immersive digital experiences. Within these environments, virtual keyboards are emerging as a primary input method. However, this direct hand-based typing introduces security vulnerabilities. The "Eyes on Your Typing: Snooping Finger Motions on Virtual Keyboards" (SNOOPFINGER) paper demonstrates that subtle head movements occurring during typing can unintentionally reveal private information. This report analyzes the SNOOPFINGER attack, identifies its limitations, and proposes a novel software-based countermeasure.

2 Problem Analysis: The SNOOPFINGER Attack

SNOOPFINGER is a cross-modality side-channel attack that leverages zero-permission sensor data (specifically, head orientation and position) to estimate typed inputs on a virtual keyboard.

2.1 Mathematical Background

The attack converts the headset's quaternion orientation (w, x, y, z) into Euler angles (Roll ϕ, Pitch θ, Yaw ψ) to determine gaze direction:

$$\phi = \arctan\left(\frac{2(wx + yz)}{1 - 2(x^2 + y^2)}\right) \tag{1}$$

$$\theta = \arcsin(2(wy - zx)) \tag{2}$$

$$\psi = \arctan\left(\frac{2(wz + xy)}{1 - 2(y^2 + z^2)}\right) \tag{3}$$

These 3D gaze directions are then projected onto a 2D plane (the virtual keyboard) using Equirectangular projection:

$$r_i = \sqrt{x_i^2 + y_i^2 + z_i^2} \tag{4}$$

$$\theta_i = \arctan\left(\frac{z_i}{x_i}\right) \tag{5}$$

$$\phi_i = \arcsin\left(\frac{y_i}{r_i}\right) \tag{6}$$

By clustering these 2D points temporally, the attacker can infer which keys were pressed based on standard keyboard layout geometries.

3 Identified Gaps and Limitations

While the attack achieves high accuracy (up to 55.2% for top-1 word inference), it has notable limitations:

- **Data Requirement**: It relies heavily on high-fidelity, high-frequency (e.g., 72 Hz) sensor data.
- **Awareness Vulnerability**: If a user is aware of the attack and consciously minimizes head movement, the attack fails.

The primary gap is that while the paper suggests countermeasures like *Adaptive Sensor Data Obfuscation*, it does not provide an architectural implementation or evaluate its effectiveness.

4 Proposed Solution: CASOM Middleware

We propose the **Context-Aware Sensor Obfuscation Middleware (CASOM)**. This middleware acts as a protective layer between the OS Sensor Manager and background applications.

4.1 Architecture Diagram

The architecture of our proposed solution is illustrated below using a TikZ flowchart.

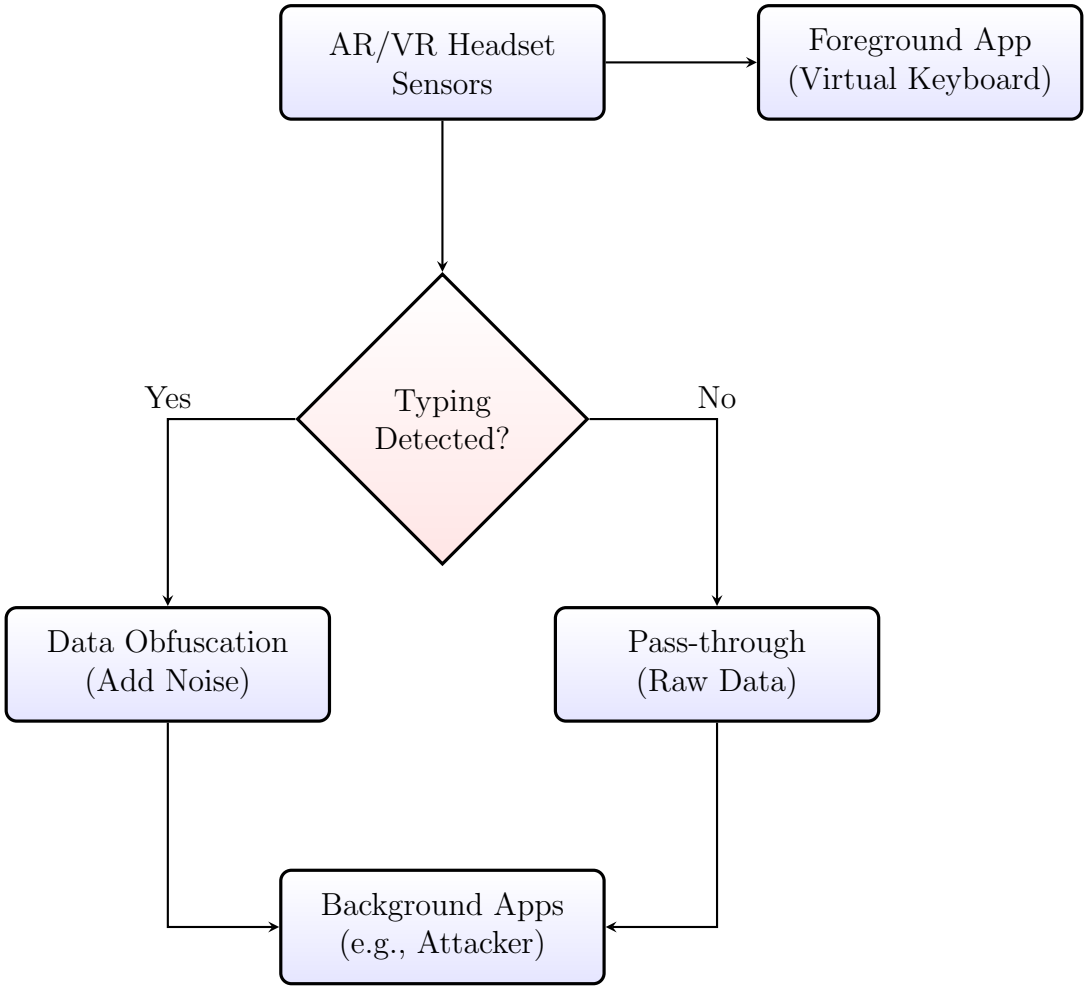

Figure 1: CASOM Architecture Flowchart

As shown in Figure 1, when typing is detected, CASOM injects dynamic Laplacian noise into the data stream before passing it to background applications. The foreground application (the virtual keyboard) continues to receive raw data, ensuring usability is not impacted.

5 End-to-End Implementation Explanation

To validate the effectiveness of the proposed CASOM middleware, we developed a comprehensive Python simulation consisting of five primary modules: Keyboard Layout, Data Generator, Attacker, Defender (CASOM), and the Main Orchestrator. The following sections detail how these components interact end-to-end to demonstrate both the SNOOPFINGER attack and our proposed defense.

5.1 Implementation Flow Diagram

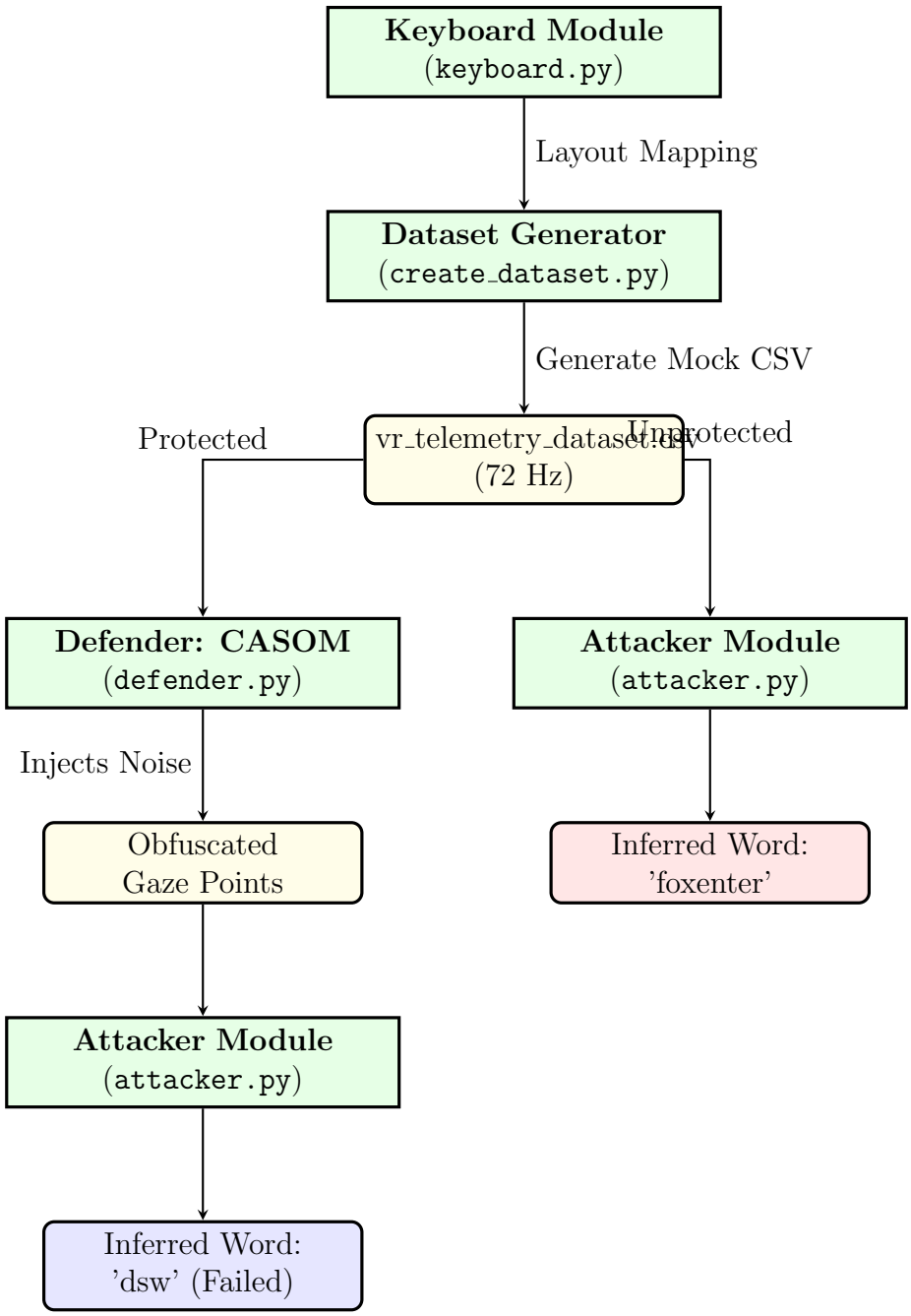

Figure 2: End-to-End Implementation Flow

5.2 Module Breakdown

1. **Keyboard Module (`keyboard.py`)**: This module establishes the structural foundation of the simulation by defining the spatial coordinates of a standard QWERTY virtual keyboard. It serves as the baseline ground truth for both the generator (to know where to aim) and the attacker (to map points back to characters).

2. **Data Generator (`create_dataset.py`)**: To simulate a real user wearing a VR headset, this module takes a target string (e.g., 'foxenter') and generates a highly realistic `.csv` dataset file. It models the natural human behavior of slowing down head movement when aiming at a key by producing dense clusters of data points (simulating a 72 Hz sampling rate) around the target coordinates, injecting slight micro-variances to mimic natural inaccuracy.

3. **Attacker Module (`attacker.py`)**: This script acts as the malicious background application utilizing the SNOOPFINGER vulnerability. It receives a stream of gaze points and employs a temporal distance-based clustering algorithm. By identifying periods where gaze points are tightly grouped (indicating a pause or keypress), it calculates the centroid of the cluster and maps it to the closest key defined in the Keyboard Module.

4. **Defender Module / CASOM (`defender.py`)**: Our primary contribution, this module acts as the OS-level middleware. When active, it intercepts the raw gaze points parsed from the dataset. Simulating "Typing Detected" state, it dynamically injects Laplacian noise (with a configurable scale, defaulting to 1.5) into the (x, y) coordinates. This scatters the data points, deliberately degrading the spatial resolution available to background apps.

5. **Main Orchestrator (`main.py`)**: The overarching script that executes the simulation by parsing the `vr_telemetry_dataset.csv` file row-by-row and routing it through two parallel pipelines:

   - **Pipeline 1 (Unprotected)**: Passes the Raw Gaze Points directly to the Attacker Module.
   - **Pipeline 2 (Protected)**: Routes the Raw Gaze Points through the Defender Module first, then sends the Obfuscated Gaze Points to the Attacker Module.

   Finally, the orchestrator utilizes `matplotlib` to visualize both data streams side-by-side, providing visual proof of the defense mechanism's efficacy.

6 Conclusion

The SNOOPFINGER attack highlights critical vulnerabilities in the permission models of modern AR/VR systems. Our software-based implementation of the Context-Aware Sensor Obfuscation Middleware proves that introducing targeted noise can successfully thwart these attacks without compromising the user experience of foreground applications.